National University of Computer and Emerging Sciences, Lahore Campus

Department of Artificial Intelligence and Data Science

CS5004: Advanced Topics in Generative AI, Fall 2026

Assignment 2: Diffusion Models, Vision Transformers and Swin

Instructor: Dr. Maryam Bashir

Instructions

- This is an **individual** assignment, designed to be completed in **about one week**. Submit on Google Classroom as announced.
- Everything runs on the **free Google Colab GPU (T4)**. No paid compute is needed.
- Several tasks are **personalised by the last digit of your roll number**, called **d** below (for example, roll number 22L-7348 gives d = 8). Using another student's settings earns zero for that part.
- A **viva** will be held. You must be able to explain every line of your code and every number in your report. Failure to attend the viva results in zero marks.
- You may use AI coding assistants for debugging, but copied code you cannot explain will be treated as plagiarism. Plagiarism or collusion results in zero marks.

CLO2: Analyse and compare transformer-based generative architectures - ViT, Swin, CLIP, Stable Diffusion, Mistral - for a given task.

Objective

In this assignment you will (A) build a small DDPM from scratch and see the forward process, training loop and sampling in action, (B) look inside pretrained ViT and Swin backbones and compare the features they learn, and (C) work through Swin dimensions and attention costs by hand. The focus is on understanding how these models work, not on chasing accuracy.

Part A: A Small Diffusion Model from Scratch (40%)

Dataset: Fashion-MNIST, padded to 32 × 32 and scaled to [−1, 1]. Train only on **your three classes**: d, (d + 3) mod 10 and (d + 6) mod 10. Each student trains on three full Fashion-MNIST classes, which is 18,000 images (6,000 per class).

Class labels: 0 T-shirt, 1 Trouser, 2 Pullover, 3 Dress, 4 Coat, 5 Sandal, 6 Shirt, 7 Sneaker, 8 Bag, 9 Ankle boot.

Rules: you may use `diffusers.UNet2DModel` as the network. The **noise schedules, forward process, training loop and sampling loop must be written by you** in PyTorch (do not use diffusers schedulers or pipelines).

A1. Forward process and noise schedules (10%)

- Implement the **linear** schedule (β from 10^{-4} to 0.02, T = 1000) and the **cosine** schedule (Nichol and Dhariwal, 2021).
- Plot $\bar{\alpha}_t$ against t for both schedules on one figure.

- Take one training image and show it at t = 0, 100, 250, 500, 750 and 999 under each schedule (a 2 × 6 grid), using the closed form $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon$.
- In 3 to 4 sentences, explain which schedule destroys the image faster and why that matters for training.

A2. Training with the ε-prediction loss (15%)

- Use a small U-Net (suggested: `block_out_channels=(32, 64, 128)`). Initialise the U-Net from a configuration with random weights; do not load pretrained weights (e.g. no `from_pretrained`).
- Implement the DDPM training step: sample x_0, t and ε, build x_t, predict $\varepsilon_\theta(x_t, t)$ and minimise $\|\varepsilon - \varepsilon_\theta\|^2$.
- Train **two models with identical settings**, one with the **linear** schedule and one with the **cosine** schedule, for about 15 minutes each on Colab.
- Plot both training loss curves on one figure and save both checkpoints. Report training time, batch size, learning rate and number of epochs.

  What to expect: samples will be recognisable and class-like, but not perfect. Some will be blurry or odd-looking, especially with less training. Samples need not be perfect; the goal is a working DDPM. Recognisable class shapes are sufficient.

A3. Sampling with DDPM (15%)

- Implement **DDPM sampling** (all 1000 reverse steps) yourself, using $x_{t-1} = (1/\sqrt{\alpha_t})\,(x_t - (\beta_t/\sqrt{(1 - \bar{\alpha}_t)})\,\varepsilon_\theta) + \sigma_t\, z$ with $\sigma_t^2 = \beta_t$.
- Generate a 4 × 4 grid of samples from **each** model, and report the sampling time.
- For one sample, show the **denoising trajectory** at 8 timesteps from x_{999} to x_0. Under each x_t, also show the model's **predicted clean image** $\hat{x}_0 = (x_t - \sqrt{(1 - \bar{\alpha}_t)}\,\varepsilon_\theta) / \sqrt{\bar{\alpha}_t}$.
- Compare the samples from the linear and cosine models. Which looks better for your classes, and does this agree with what you saw in A1? Also explain why sampling takes 1000 network evaluations while one training step needs only one.

Part B: Inside ViT and Swin (40%)

Dataset (your own): photograph **4 classes** of everyday objects around you (for example mugs, shoes, books, plants), **25 photos per class**, 100 photos in total, taken with your phone. Vary the background, angle and lighting. Include a collage of your dataset in the report. These photos must be your own; the viva will check this.

Models: pretrained `vit_base_patch16_224` and `swin_tiny_patch4_window7_224` from `timm`. No fine-tuning is required.

Preprocessing your photos: do not resize by hand. Use each model's own timm transform: `cfg = timm.data.resolve_model_data_config(model)` and `transform = timm.data.create_transform(**cfg, is_training=False)`. It resizes the shorter side, centre-crops 224 × 224 and normalises with ImageNet statistics. Load each photo with `ImageOps.exif_transpose(Image.open(path)).convert('RGB')` so phone photos are not rotated sideways. Keep the object centred and filling most of the frame, since the centre crop removes the edges.

B1. Trace the shapes (15%)

Note: both models are ImageNet **classification** models. B1 inspects only their **backbone** (patch embedding and stages/blocks), which is the same backbone the Swin paper reuses for detection (Cascade Mask R-CNN) and segmentation (UperNet); only the head on top changes. **Hint:** `timm.create_model('swin_tiny_patch4_window7_224', pretrained=True, features_only=True)` removes the classification head and returns the four stage outputs directly, exactly as a detection or segmentation backbone uses them.

- Use forward hooks to print the output shape after the patch embedding and after every stage (Swin) or every 3rd block (ViT) for one 224 × 224 image.
- Present the results in a table showing the number of tokens, the channel width, and where downsampling happens.
- Report the parameter count of each model. In 2 to 3 sentences, explain why Swin's feature maps can feed an FPN detector directly while ViT's cannot.

B2. Linear probe on your photos (25%)

- Freeze both backbones. Extract one feature vector per image (the [CLS] token for ViT, global average pooling for Swin).
- Train a logistic regression classifier on each set of features using a stratified 70/30 split (same split for both).
- Report accuracy and macro-F1, show both confusion matrices and display 3 misclassified images per model (or all, if fewer). Discuss which backbone gave better features for your data and suggest why.

Part C: Working by Hand (20%)

Answer in the report, showing all working. Use the settings for your digit d:

d	Image size (H × W × 3)	C
0	224 × 224	64
1	224 × 224	96
2	224 × 224	128
3	448 × 448	64
4	448 × 448	96
5	448 × 448	128
6	672 × 672	64
7	672 × 672	96
8	672 × 672	128
9	896 × 896	96

Assume patch size 4, window size M = 7 and stage depths 2, 2, 6, 2.

- Give the output size (h × w × channels) of the linear embedding and of each of the 4 stages.
- Give the number of windows in each stage.

- For one attention layer in stage 1, compute $\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$ and $\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$, and the ratio between them.

Deliverables

- **One Colab notebook** (`RollNo_A2.ipynb`) with all cells executed, so it does not need to be re-run.
- **One report** (PDF, at most 8 pages) containing the figures, tables, discussion and Part C working.
- The trained Part A checkpoint and your Part B photos, uploaded to a Google Drive folder, with the link given in the report.

Marks Distribution

Part	Task	Marks
A	A1 Forward process and noise schedules	5
A	A2 Training with the ε-prediction loss	8
A	A3 Sampling with DDPM	8
B	B1 Trace the shapes	8
B	B2 Linear probe on your photos	12
C	Swin dimensions and attention cost (by hand)	10
Total		51